Análise léxica

O que vocês fizeram à mão, e a ferramenta que faz

Encontro 8 | 16 de setembro

Linguagens Formais e Compiladores

O que vocês construíram na E1

Um autômato para cada tipo de token: identificador, número, operador, palavra-chave.

O que vocês construíram na E1

Um autômato para cada tipo de token: identificador, número, operador, palavra-chave.

E um programa que lia a entrada e decidia qual autômato tinha casado.

O que vocês construíram na E1

Um autômato para cada tipo de token: identificador, número, operador, palavra-chave.

E um programa que lia a entrada e decidia qual autômato tinha casado.

Hoje damos nome às partes, e vemos a ferramenta que gera tudo isso.

1
Duas fases, dois nomes

Como você lê esta frase

o gato dorme

Como você lê esta frase

`o gato dorme`

Você não lê letra por letra procurando sentido.

Como você lê esta frase

`o gato dorme`

Você não lê letra por letra procurando sentido.

Primeiro junta letras em **palavras**. Depois junta palavras em **frase**.

Como você lê esta frase

```
o gato dorme
```

Você não lê letra por letra procurando sentido.

Primeiro junta letras em **palavras**. Depois junta palavras em **frase**.

São duas etapas diferentes, e o compilador faz igual.

As duas fases



As duas fases



análise léxica agrupa caracteres em palavras

análise sintática agrupa palavras em estrutura

Os nomes em inglês

As ferramentas são todas em inglês, e usam outros nomes para as mesmas duas coisas.

em português	em inglês	o programa
análise léxica	*lexical analysis*	**lexer**
análise sintática	*syntactic analysis*	**parser**

Os nomes em inglês

As ferramentas são todas em inglês, e usam outros nomes para as mesmas duas coisas.

em português	em inglês	o programa
análise léxica	*lexical analysis*	**lexer**
análise sintática	*syntactic analysis*	**parser**

Lexer e ***parser***. São essas as palavras que vocês vão ver na documentação e nas mensagens de erro.

Por que separar em duas?

Nada impede, em princípio, fazer tudo de uma vez.

Por que separar em duas?

Nada impede, em princípio, fazer tudo de uma vez.

Mas os dois problemas têm *naturezas diferentes*.

Por que separar em duas?

Nada impede, em princípio, fazer tudo de uma vez.

Mas os dois problemas têm *naturezas diferentes*.

E vocês descobriram isso na aula passada.

A razão, em uma tabela

	lexer	**parser**
reconhece	palavras isoladas	estrutura aninhada
precisa de	memória finita	uma pilha
a linguagem é	regular	livre de contexto
a máquina é	autômato finito	autômato de pilha

A razão, em uma tabela

	lexer	**parser**
reconhece	palavras isoladas	estrutura aninhada
precisa de	memória finita	uma pilha
a linguagem é	regular	livre de contexto
a máquina é	autômato finito	autômato de pilha

A divisão do compilador em duas fases espelha exatamente a divisão entre as duas classes de linguagem.

E há uma razão prática

Autômato finito é **muito** mais rápido que pilha.

E há uma razão prática

Autômato finito é **muito** mais rápido que pilha.

O lexer resolve com máquina barata tudo o que dá,
e entrega ao parser só o que exige máquina cara.

E há uma razão prática

Autômato finito é **muito** mais rápido que pilha.

O lexer resolve com máquina barata tudo o que dá,
e entrega ao parser só o que exige máquina cara.

Um arquivo de mil linhas tem dezenas de milhares de caracteres,
e talvez mil tokens. O lexer reduz o problema em vinte vezes.

2
O lexer por dentro

O que entra e o que sai



O que entra e o que sai



entra uma sequência de caracteres

sai uma sequência de *tokens*

O que é um token

DEFINIÇÃO • TOKEN

Um par: o **tipo** e o **texto** que foi reconhecido.

O que é um token

DEFINIÇÃO • TOKEN

Um par: o **tipo** e o **texto** que foi reconhecido.

tipo	texto
SE	se
IDENT	x
IGUAL	==
INTEIRO	10

O que é um token

DEFINIÇÃO • TOKEN

Um par: o **tipo** e o **texto** que foi reconhecido.

tipo	texto
SE	se
IDENT	x
IGUAL	==
INTEIRO	10

O tipo é o que o analisador sintático usa. O texto é o que o resto do compilador guarda.

Cada tipo de token é uma linguagem regular

`IDENT`	letra, depois letras ou dígitos
`INTEIRO`	um ou mais dígitos
`REAL`	dígitos, ponto, dígitos

Cada tipo de token é uma linguagem regular

`IDENT`	letra, depois letras ou dígitos
`INTEIRO`	um ou mais dígitos
`REAL`	dígitos, ponto, dígitos

Todas descritíveis por expressão regular.
E toda expressão regular vira autômato finito.

Cada tipo de token é uma linguagem regular

`IDENT`	letra, depois letras ou dígitos
`INTEIRO`	um ou mais dígitos
`REAL`	dígitos, ponto, dígitos

Todas descritíveis por expressão regular.
E toda expressão regular vira autômato finito.

É por isso que a análise léxica cabe na unidade que acabamos de estudar.

3
As duas decisões

Um problema que vocês enfrentaram

A entrada é `sela`. O autômato de `SE` casa com os dois primeiros caracteres.

Um problema que vocês enfrentaram

A entrada é `sela`. O autômato de `SE` casa com os dois primeiros caracteres.

O analisador deve emitir `SE` e continuar do `l`?

Um problema que vocês enfrentaram

A entrada é `sela`. O autômato de `SE` casa com os dois primeiros caracteres.

O analisador deve emitir `SE` e continuar do `l`?

Não. E a regra que resolve isso é a primeira das duas que todo analisador léxico segue.

Decisão 1: casamento mais longo

Entre todos os padrões que casam a partir da posição atual,
escolhe-se o que consome **mais caracteres**.

Decisão 1: casamento mais longo

Entre todos os padrões que casam a partir da posição atual, escolhe-se o que consome **mais caracteres**.

entrada	resultado	por quê
`sela`	`IDENT`	`IDENT` casa 4, `SE` casa 2
`3.14`	`REAL`	`REAL` casa 4, `INTEIRO` casa 1
`==`	`IGUAL`	`IGUAL` casa 2, `ATRIB` casa 1

Decisão 1: casamento mais longo

Entre todos os padrões que casam a partir da posição atual, escolhe-se o que consome **mais caracteres**.

entrada	resultado	por quê
`sela`	`IDENT`	`IDENT` casa 4, `SE` casa 2
`3.14`	`REAL`	`REAL` casa 4, `INTEIRO` casa 1
`==`	`IGUAL`	`IGUAL` casa 2, `ATRIB` casa 1

Verificado numa simulação: os três saem assim.

Mas e quando empata?

A entrada é se. Os dois padrões casam **dois** caracteres:
`SE` e `IDENT`.

Mas e quando empata?

A entrada é se. Os dois padrões casam **dois** caracteres:
`SE` e `IDENT`.

O comprimento não resolve. Precisa de outro critério.

Decisão 2: a ordem de declaração

No empate, vence o padrão declarado **primeiro**.

Decisão 2: a ordem de declaração

No empate, vence o padrão declarado **primeiro**.

`SE` antes de `IDENT`	se vira `SE`
`IDENT` antes de `SE`	se vira `IDENT`

Decisão 2: a ordem de declaração

No empate, vence o padrão declarado **primeiro**.

`SE` antes de `IDENT`	se vira `SE`
`IDENT` antes de `SE`	se vira `IDENT`

Se a ordem estiver errada, a palavra-chave nunca é reconhecida.

Decisão 2: a ordem de declaração

No empate, vence o padrão declarado **primeiro**.

`SE` antes de `IDENT`	se vira `SE`
`IDENT` antes de `SE`	se vira `IDENT`

Se a ordem estiver errada, a palavra-chave nunca é reconhecida.

Palavras-chave sempre antes do identificador. Sempre.

A ordem das duas regras importa

O `ATRIB` (=) foi declarado *antes* do `IGUAL` (==).

A ordem das duas regras importa

O `ATRIB` (=) foi declarado *antes* do `IGUAL` (==).

Ainda assim, a entrada == vira `IGUAL`.

A ordem das duas regras importa

O `ATRIB` (=) foi declarado *antes* do `IGUAL` (==).

Ainda assim, a entrada == vira `IGUAL`.

Porque o comprimento decide *antes* da ordem.
A ordem só desempata.

Um programa inteiro

```
se x == 10 entao y = 3.5
```

SE	se
IDENT	x
IGUAL	==
INTEIRO	10
IDENT	entao
IDENT	y
ATRIB	=
REAL	3.5

Um programa inteiro

```
se x == 10 entao y = 3.5
```

```
SE        se
IDENT     x
IGUAL     ==
INTEIRO   10
IDENT     entao
IDENT     y
ATRIB     =
REAL      3.5
```

O `entao` virou `IDENT` porque esqueci de declará-lo como palavra-chave. O analisador não reclama.

4
O gerador

Ninguém escreve isso à mão

Vocês escreveram na E1, e foi bom para entender.
Mas escalar isso para uma linguagem real é inviável.

Ninguém escreve isso à mão

Vocês escreveram na E1, e foi bom para entender.
Mas escalar isso para uma linguagem real é inviável.

Existem ferramentas que *geram* o analisador a partir das expressões regulares.

Ninguém escreve isso à mão

Vocês escreveram na E1, e foi bom para entender.
Mas escalar isso para uma linguagem real é inviável.

Existem ferramentas que *geram* o analisador a partir das expressões regulares.

As clássicas são `lex` e `flex`, que geram C.
A que vamos usar é o **ANTLR**, que gera Python.

O que o ANTLR faz



O que o ANTLR faz



Você escreve **um arquivo** com as regras.
Ele gera as classes que fazem a análise.

O que o ANTLR faz



Você escreve **um arquivo** com as regras.
Ele gera as classes que fazem a análise.

Não é preciso escrever autômato nem pilha.

A ideia central do arquivo

nome	é regra de	roda em	classe
`NUMERO`, `IDENT`	lexer	autômato finito	regular
`expr`, `instrucao`	parser	pilha	livre de contexto

A ideia central do arquivo

nome	é regra de	roda em	classe
`NUMERO`, `IDENT`	lexer	autômato finito	regular
`expr`, `instrucao`	parser	pilha	livre de contexto

A caixa da primeira letra diz qual máquina executa a regra.

A ideia central do arquivo

nome	é regra de	roda em	classe
`NUMERO`, `IDENT`	lexer	autômato finito	regular
`expr`, `instrucao`	parser	pilha	livre de contexto

A caixa da primeira letra diz qual máquina executa a regra.

Maiúscula não pode ser recursiva: autômato finito não tem pilha.

Uma gramática léxica completa

```
grammar Calc;
programa : .*? EOF ; // deixa para a E3

SE : 'se' ;
ENQUANTO: 'enquanto' ;
IDENT : [a-zA-Z_] [a-zA-Z_0-9]* ;
REAL : [0-9]+ '.' [0-9]+ ;
INTEIRO : [0-9]+ ;
IGUAL : '==' ;
ATRIB : '=' ;
COMENT : '//' ~[\r\n]* -> skip ;
ESPACO : [ \t\r\n]+ -> skip ;
```

Uma gramática léxica completa

```
grammar Calc;
programa : .*? EOF ; // deixa para a E3

SE : 'se' ;
ENQUANTO: 'enquanto' ;
IDENT : [a-zA-Z_] [a-zA-Z_0-9]* ;
REAL : [0-9]+ '.' [0-9]+ ;
INTEIRO : [0-9]+ ;
IGUAL : '==' ;
ATRIB : '=' ;
COMENT : '//' ~[\r\n]* -> skip ;
ESPACO : [ \t\r\n]+ -> skip ;
```

As palavras-chave vêm antes de `IDENT`. Não é estilo: é necessário.

Os operadores das regras de lexer

escreve-se	significa	exemplo
`'abc'`	exatamente esse texto	`'se'`
`[abc]`	um dos caracteres	`[+-*/]`
`[a-z]`	faixa	`[0-9]`
`~[abc]`	qualquer *exceto* esses	`~['\n]`
`* + ?`	zero ou mais, uma ou mais, opcional	
`|`	ou	`'sim' | 'nao'`
`-> skip`	reconhece e descarta	

Os operadores das regras de lexer

escreve-se	significa	exemplo
`'abc'`	exatamente esse texto	`'se'`
`[abc]`	um dos caracteres	`[+-*/]`
`[a-z]`	faixa	`[0-9]`
`~[abc]`	qualquer *exceto* esses	`~['\n]`
`* + ?`	zero ou mais, uma ou mais, opcional	
`|`	ou	`'sim' | 'nao'`
`-> skip`	reconhece e descarta	

São os mesmos operadores do encontro 4, com notação um pouco diferente.

O `-> skip`

Diz: *reconheça isto e jogue fora*.

O -> skip

Diz: *reconheça isto e jogue fora*.

**O token existe para o lexer saber onde termina,
mas não é entregue ao parser.**

O `-> skip`

Diz: *reconheça isto e jogue fora*.

**O token existe para o lexer saber onde termina,
mas não é entregue ao parser.**

É assim que se descarta espaço e comentário.
Sem ele, toda regra do parser teria de tratar espaços.

5

Mãos à obra

Instalando

```
pip install antlr4-tools antlr4-python3-runtime
```

Instalando

```
pip install antlr4-tools antlr4-python3-runtime
```

O primeiro cria os comandos `antlr4` e `antlr4-parse`, que baixam o Java sozinhos na primeira execução.

Instalando

```
pip install antlr4-tools antlr4-python3-runtime
```

O primeiro cria os comandos `antlr4` e `antlr4-parse`,
que baixam o Java sozinhos na primeira execução.

Instalem agora, nesta aula. Não deixem para a véspera.

Testando sem gerar código

```
antlr4-parse Calc.g4 programa -tokens
```

Digite o programa, encerre com `Ctrl+D`, e ele lista os tokens.

Testando sem gerar código

```
antlr4-parse Calc.g4 programa -tokens
```

Digite o programa, encerre com `Ctrl+D`, e ele lista os tokens.

Este é o comando mais útil da E2.

Testando sem gerar código

```
antlr4-parse Calc.g4 programa -tokens
```

Digite o programa, encerre com `Ctrl+D`, e ele lista os tokens.

Este é o comando mais útil da E2.

Testem cada regra assim que escreverem.
Escrever a gramática inteira antes de testar é a receita para não achar o erro.

Gerando o código

```
antlr4 -Dlanguage=Python3 -visitor -o gerado gramatica/Calc.g4
```

`-Dlanguage=Python3`	gera Python em vez de Java
`-visitor`	gera também a classe de visita
`-o gerado`	põe a saída nesse diretório

Gerando o código

```
antlr4 -Dlanguage=Python3 -visitor -o gerado gramatica/Calc.g4
```

`-Dlanguage=Python3`	gera Python em vez de Java
`-visitor`	gera também a classe de visita
`-o gerado`	põe a saída nesse diretório

O `-visitor` só será necessário na E4. Não custa gerar desde já.

Duas coisas para o repositório

não versione	o diretório `gerado/`
versione	o `.g4` e o `gerar.sh`

Duas coisas para o repositório

não versione	o diretório `gerado/`
versione	o `.g4` e o `gerar.sh`

O código gerado é reproduzível. Commitá-lo polui o histórico e leva alguém a editar o arquivo errado.

Duas coisas para o repositório

não versione	o diretório `gerado/`
versione	o `.g4` e o `gerar.sh`

O código gerado é reproduzível. Commitá-lo polui o histórico e leva alguém a editar o arquivo errado.

O esqueleto que passei já vem com o `.gitignore` certo.

Os erros que vocês vão cometer

sintoma	**causa**
palavra-chave vira `IDENT`	a regra dela está depois de `IDENT`
o parser reclama de espaço	faltou o `-> skip`
`ModuleNotFoundError` no import	o `gerar.sh` não rodou
erro estranho de versão	o jar e o runtime não batem
regra nunca dispara	outra anterior casa o mesmo texto

Os erros que vocês vão cometer

sintoma	causa
palavra-chave vira `IDENT`	a regra dela está depois de `IDENT`
o parser reclama de espaço	faltou o `-> skip`
`ModuleNotFoundError` no import	o `gerar.sh` não rodou
erro estranho de versão	o jar e o runtime não batem
regra nunca dispara	outra anterior casa o mesmo texto

O primeiro é o mais comum, e o mais difícil de perceber:
não dá erro, só passa a reconhecer errado.

O que fica

- O lexer recebe caracteres e devolve tokens. O parser recebe tokens e monta a estrutura.

O que fica

- O lexer recebe caracteres e devolve tokens. O parser recebe tokens e monta a estrutura.
- Token é um par: o tipo e o texto reconhecido.

O que fica

- O lexer recebe caracteres e devolve tokens. O parser recebe tokens e monta a estrutura.
- Token é um par: o tipo e o texto reconhecido.
- Cada tipo de token é uma linguagem regular, logo cabe num autômato.

O que fica

- O lexer recebe caracteres e devolve tokens. O parser recebe tokens e monta a estrutura.
- Token é um par: o tipo e o texto reconhecido.
- Cada tipo de token é uma linguagem regular, logo cabe num autômato.
- Decisão 1: vence o casamento mais longo.

O que fica

- O lexer recebe caracteres e devolve tokens. O parser recebe tokens e monta a estrutura.
- Token é um par: o tipo e o texto reconhecido.
- Cada tipo de token é uma linguagem regular, logo cabe num autômato.
- Decisão 1: vence o casamento mais longo.
- Decisão 2: no empate, vence quem foi declarado antes.

O que fica

- O lexer recebe caracteres e devolve tokens. O parser recebe tokens e monta a estrutura.
- Token é um par: o tipo e o texto reconhecido.
- Cada tipo de token é uma linguagem regular, logo cabe num autômato.
- Decisão 1: vence o casamento mais longo.
- Decisão 2: no empate, vence quem foi declarado antes.
- Por isso palavras-chave vêm antes do identificador.

O que fica

- O lexer recebe caracteres e devolve tokens. O parser recebe tokens e monta a estrutura.
- Token é um par: o tipo e o texto reconhecido.
- Cada tipo de token é uma linguagem regular, logo cabe num autômato.
- Decisão 1: vence o casamento mais longo.
- Decisão 2: no empate, vence quem foi declarado antes.
- Por isso palavras-chave vêm antes do identificador.
- E o ANTLR gera tudo isso a partir de um arquivo `.g4`.

O que fica

- O lexer recebe caracteres e devolve tokens. O parser recebe tokens e monta a estrutura.
- Token é um par: o tipo e o texto reconhecido.
- Cada tipo de token é uma linguagem regular, logo cabe num autômato.
- Decisão 1: vence o casamento mais longo.
- Decisão 2: no empate, vence quem foi declarado antes.
- Por isso palavras-chave vêm antes do identificador.
- E o ANTLR gera tudo isso a partir de um arquivo `.g4`.

Agora: escrevam a gramática léxica do projeto de vocês.